Written Reflection

1. What did you delegate to AI, and how did you provide context (files, instructions, verification steps)?

I use Codex to help complete the coding work, include the project initializing, DB models, API implementation, FE development and Auto testing.
First step is providing the assessment file/doc to AI and let it analyze the task. In this request, I will provide some limitation such as language, structure, DB if not mentioned in doc.
Second step, I will review the analyze and separate the whole task into several steps. This can help me understand everything, figure out is any necessary limitation I missed and design the whole workflow. After get it, I will generate the Agent file for AI(If exist is better), and supply the each minor task to AI for coding.
After each task, enforce AI to run auto testing and I will do manually testing to make sure this step is completed fine.
If all things done, re-run all testing, check the function in local environment. If in real dev, I will push to dev/qa for smoke and integration test.

2. Where did the AI's output diverge from the requirements or from your own intent? How did you notice, and what did you change?

In this Certificate service, the AI output is good and meet my requirement in each step requirement. I prefer to do the manually testing after each function development, especially the edge case, such as the empty/space search, re-revoke certificate, re bulk issuance. Make sure multi command won't change the DB.
In final step to update README and start instruction. I manually test and find the AI combine the example seed with start service. I request it separate them, start service won't auto seed the DB

Design questions
1. Issuing a certificate must also write an audit-log entry and send a notification. The API endpoint and a background job can both be retried and can run concurrently. How do you guarantee no learner is notified twice?

The DB use the unique constraints, DB transaction and the Transaction Outbox pattern, when issueing a certificate, the system creates the certificate, audit log and notification with this exact transaction. The unique contraint prevents the concurrent request from issuing duplicate certificate and create duplicate notification events.

The outbox event will use the unique idempotency key.this can guarantee the issue won't be noticed twice.

2. Certificates must be sorted and exported by tenant-local calendar date. Timestamps are stored in UTC. A colleague proposes storing one fixed UTC offset per tenant (Sydney +10, Perth +8) and converting with that offset everywhere. Sydney observes daylight saving time;

Perth does not. What goes wrong with this proposal, and how would you implement it instead?

Use the fixed UTC time zone would be incorrect. So I would store all timezone as UTC and save their IANA timezone identification, such as Australiz/Sydney, to help calculate the real local time when user look it in local time.

And this is easier to help sort/filter. All data time zone is same and we are easily to implement the current sort/filter function and future expansion.

3. A single tenant grows to 100,000 certificates and the CSV export times out. Walk me through your fix

I will choose to use the pagination method. Separate the whole 100,000 certificates to different smaller task. Such as separate it into 100 request and combine them as batch request. And push it to background worker to handle the request. The separation can make sure the memory won't be broken. The background worker can help release the service to handle other request when the CSV export request needs a long time.